SIMATS ENGINEERING

Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA3715**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

*I **B.Hari Sai/192311098** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Hari Sai

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWERS:

Q1. Online Banking Fund Transfer Failures

1.The failures occur mainly at minimum and maximum transfer limits, suggesting boundary-handling defects, incorrect comparison operators, rounding issues, or inconsistent validation between the interface and server. The test strategy should target these limits rather than depend mainly on random inputs.

2. Equivalence Partitioning (ECP)

Divide transfer amounts into meaningful classes, assuming the system has a minimum limit L and maximum limit U:

Below minimum: amount $< L$ — reject.

Valid range: $L \leq$ amount $\leq U$ — accept if all other checks pass.

Above maximum: amount $> U$ — reject.

Also test invalid formats, zero, negative values, blank input, and excessive decimal places where applicable.

3. Boundary Value Analysis (BVA)

Test values immediately around both limits: $L-1$ smallest currency unit, L, $L+1$ unit; $U-1$ unit, U, $U+1$ unit. Include values such as zero and negative amounts if permitted by the input format. Verify the displayed balance, transaction status, ledger entry, and error message.

4. Random testing assessment

Random testing can expose unexpected combinations, but it may rarely select exact boundaries and is difficult to reproduce unless the random seed and data are recorded. Retain it as supplementary fuzz/random testing, not as the primary design.

5. Optimized approach

Use ECP and BVA as the core; add decision-table tests for account status, available balance, daily limit, beneficiary status, and authentication. Include negative, duplicate-submit, timeout, and retry cases. Automate regression tests and trace each requirement to cases. This gives repeatable boundary-focused coverage while preserving random testing's value for unexpected inputs.

Q2. Healthcare Module: White-Box Coverage Gaps

1.Critical post-deployment defects indicate that the tests did not exercise important logic paths or combinations. A test report showing a high percentage alone does not prove that safety-relevant behavior was adequately tested.

2. Conceptual control flow graph (CFG)

Model the module using nodes for entry, input validation, patient/record checks, clinical-rule evaluation, result generation, error handling, and exit. Connect nodes for both true and

false outcomes of each decision. Include exception and invalid-data edges where the code has them.

Example conceptual paths:

Entry → valid input → valid patient record → rule passes → result saved → exit.

Entry → invalid input → error response → exit.

Entry → valid input → missing/inconsistent record → error handling → exit.

Entry → valid input → rule fails → alert/exception path → exit.

3. Coverage analysis

Statement coverage only confirms that statements ran at least once. Branch coverage checks each decision outcome. Path coverage checks sequences of decisions, but complete path coverage may be infeasible where loops create infinitely many paths. A healthcare module should prioritize branch coverage and clinically significant paths, plus exception handling.

4. Improved strategy

Build the CFG from the source, identify independent paths using cyclomatic complexity, and design tests for each feasible basis path. Add boundary, invalid-data, missing-record, exception, and integration tests. Use risk-based prioritization for clinical decisions and maintain traceability from requirements to tests.

This strategy reveals untested decision outcomes and failure handling, not merely executed lines. It improves confidence and reduces risk, but testing should be combined with code review, integration testing, and appropriate clinical validation.

Q3. E-Commerce Price, Quantity and Discount Combinations

1.Testing each field independently can miss defects caused by interactions among price, quantity, and discount—for example, a discount applied twice, a negative total, or an incorrect rounding result.

2. Equivalence classes and boundaries

For price, test invalid negative values, zero if not allowed, valid prices, and values at the minimum/maximum supported price. For quantity, test negative, zero, one, typical quantity, and the maximum allowed quantity. For discount, test below zero, valid discount range, maximum permitted discount, and above the permitted maximum. Confirm the actual rules before assigning exact numeric boundaries.

3. Decision table

Create columns for combinations such as valid/invalid price, valid/invalid quantity, and valid/invalid discount. Expected actions should include calculate total, reject input, or display a validation message. Add business rules for maximum discount, eligible products, and quantity-based discounts if applicable.

4. Other black-box methods

Use ECP to select representative values from each class and BVA for edge values. Use pairwise testing for broad interaction coverage when exhaustive combinations are too large,

and targeted tests for high-risk combinations. Check rounding, currency precision, tax order, and whether discount applies before or after tax.

5. Revised approach

Start with requirement-based decision tables, then add ECP/BVA cases and pairwise combinations. Compare calculated totals with an independently calculated expected result. Automate regression cases for discovered defects. This balances coverage and execution cost without claiming that every possible combination has been tested.

Q4. Nested Conditions and Loops: Path Coverage

1.Partial path coverage may leave combinations of nested conditions, loop entry/exit behavior, and error branches untested. If the CFG is incomplete, the team may not recognize important independent paths.

2. CFG and independent paths

Represent each decision as a node with true and false edges, each loop with entry, repetition, and exit edges, and each exception or early return explicitly. Check that all feasible control transfers are represented. Use cyclomatic complexity, commonly $V(G) = E - N + 2P$ (for P connected components; usually P=1), or decisions + 1 for a single connected structured graph, to estimate the number of linearly independent basis paths.

3. Testing strategy

Choose a basis set of feasible paths and create inputs that force each decision outcome. For loops, test zero iterations, one iteration, multiple iterations, and values near the maximum supported count. For nested conditions, construct tests that reach each true/false branch and important combinations. Include early termination and exception paths.

Basis path testing covers a linearly independent set of paths, not every possible execution path. Loops can make total path coverage infinite or impractical. Prioritize feasible, safety- or business-critical paths and supplement with branch/condition coverage, boundary tests, and targeted integration tests. This provides a measurable, defensible approach to improving defect detection.

Q5. Financial Application: Hybrid Testing Strategy

1.Defects persisting in production suggest gaps in requirements coverage, boundary handling, internal decision logic, integration, or regression testing. Using both black-box and white-box testing is not sufficient unless their objectives and adequacy criteria are explicit.

2. Evaluate adequacy

Review requirement-to-test traceability, defect escape trends, statement/branch coverage, risk coverage, and whether edge cases and error paths are included. Coverage percentages should be interpreted alongside test quality; high statement coverage alone does not establish correctness.

3. Hybrid design

Black-box layer: apply ECP and BVA to transaction amounts, fees, interest, limits, dates, and account inputs. Use decision tables for combinations such as account type, transaction status, balance, and fee eligibility.

White-box layer: construct CFGs for critical calculations and authorization logic, test branches and independent basis paths, and inspect exception handling and loop boundaries.

4. Integration and regression

Test API/database interactions, transaction atomicity, concurrent requests, timeout/retry behavior, duplicate submissions, rollback, audit logs, and reconciliation. Add regression tests for every production defect and run them in the release pipeline.

Black-box tests validate externally visible requirements; white-box tests expose unexecuted internal logic. Risk-based prioritization and traceability connect the two. Together with code review, security testing, and production monitoring, this reduces defect escape more effectively than relying on either method alone.

Q6. HR Payroll Bonus Errors at 5, 10 and 15 Years

1.Incorrect payouts at exactly 5, 10, and 15 years strongly suggest threshold logic errors—for example, using "greater than" instead of "greater than or equal to," incorrect rounding, or overlapping service bands. The actual bonus policy must be confirmed before defining expected payouts.

2. Equivalence Partitioning

Partition service years into policy-defined bands, such as below 5, 5 to below 10, 10 to below 15, and 15 or above, only if these match the payroll rules. Partition performance ratings into each valid rating category and invalid/out-of-range values. Test representative values from every class.

3. Boundary Value Analysis

For each threshold, test immediately below, exactly at, and immediately above it. If service is stored in completed years, use 4/5/6, 9/10/11, and 14/15/16 years; if stored as dates or fractional years, use dates or values just before, on, and after the anniversary. Cross these with relevant performance ratings.

4. Why more effective than exploratory testing alone

Exploratory testing can discover unexpected issues but does not guarantee systematic testing of every threshold. ECP and BVA directly target the known risk and produce repeatable, auditable evidence.

5. Validation

Independently calculate expected bonuses from the approved policy, verify rounding and currency precision, and check payroll reports and audit records. Add each defect as a permanent regression test. Do not assume a particular payout formula without the official rules.

Q7. Laboratory Temperature Control and MC/DC

1.An alarm failing during a sensor-malfunction sequence is a safety concern. Branch coverage may be insufficient when several Boolean conditions jointly determine whether the heater, cooler, or alarm activates.

2. Conceptual CFG

A suitable conceptual flow is: read sensors → validate sensor status → decide whether temperature is above the upper threshold → control cooler; decide whether below the lower threshold → control heater; evaluate sensor fault or implausible readings → activate alarm/safe state → log event → repeat/exit. Each decision should have true and false edges. The actual CFG must be derived from the implementation.

3. MC/DC strategy

For each compound decision, identify every atomic condition. Construct test pairs where one condition changes while all other conditions remain fixed, and that change independently changes the decision outcome. Include normal sensor values, high and low temperatures, sensor disconnected, stale reading, out-of-range reading, and combinations of a fault with a temperature threshold.

4. Safety behavior

Verify fail-safe behavior: on unreliable sensor data, the system should enter the approved safe state and raise the alarm. Confirm alarm timing, logging, recovery, and that heater and cooler commands do not conflict.

MC/DC demonstrates that each condition can independently affect a decision and is stronger than statement coverage for complex Boolean logic. It supports safety assurance but does not alone prove system safety; requirements review, hardware-in-the-loop testing, fault injection, and independent validation are also needed.

Q8. Travel Portal: Promo-Code Combinations

1.Testing promo codes one at a time misses interactions among base price, promotion type, user tier, stacking rules, and discount order. A negative total suggests missing validation or incorrect discount arithmetic.

2. Decision-table design

List conditions: price is valid; user tier is eligible; promo code is valid; promotion types are compatible; stacking is allowed; discount cap is respected. Define actions: apply discount, reject the code, allow only one code, or show a clear message. Include rules for zero/negative price, expired codes, duplicate codes, incompatible promotions, and discounts exceeding the payable amount.

3. Cause-effect mapping

Map each input condition to outcomes such as “promotion accepted,” “discount capped,” or “promotion rejected.” Verify the resulting total never becomes negative and follows the

approved business rules. Exact expected results depend on the portal's actual promotion policy.

4. Optimized test set

Use one test for each distinct decision-table rule, plus boundary values for price and discount caps. Add targeted combinations for each user tier and promo type, including valid stacking and prohibited stacking. If the full combination space is large, use pairwise selection for routine combinations and retain exhaustive tests for high-risk rules.

Decision tables make interactions explicit and remove redundant cases with identical outcomes. Pairwise testing reduces the number of routine combinations, while risk-based tests preserve coverage of critical stacking and negative-total conditions.

Q9. Database Migration: Infinite-Loop Crashes

1.High statement coverage does not ensure loops terminate. The code may execute every statement while repeating indefinitely for certain records, malformed data, or boundary values.

2. CFG and path analysis

Model nested loops, validation branches, error exits, and loop updates in the CFG. Calculate cyclomatic complexity $V(G) = E - N + 2P$, or decisions + 1 for a single connected structured graph, to estimate a basis-path set. Use this metric as a guide, not as proof that every loop behavior is covered.

3. Loop-boundary tests

For each loop, test zero records, one record, multiple records, exactly the supported maximum, and maximum plus one where meaningful. For nested loops, test combinations where inner loops run zero, one, and multiple times. Include malformed records, duplicate keys, missing fields, and exceptions during validation.

4. Termination controls

Check that loop variables progress on every continuing path, termination conditions can become true, and retries have a bound. Add timeouts or watchdogs where appropriate, logging of record identifiers and iteration counts, and tests for cancellation/recovery. Avoid changing business logic without confirming the intended behavior.

Basis-path testing targets independent control-flow behavior, while loop-boundary tests focus on repetition and termination. Statement coverage alone misses these properties. Combine them with large-data performance tests, migration reconciliation, and regression tests for every crash.

Q10. Food Delivery Tracker: State Transition Testing

1.The tracker is a dynamic workflow. A driver cancellation during delivery may leave the order in a state with no valid outgoing transition, so testing isolated input values is insufficient.

2. State model

Define states such as Ordered, Preparing, Out for Delivery, Delivered, Cancelled, and Reassignment Pending. Confirm the actual business rules for whether cancellation is allowed after pickup and whether reassignment is possible. Define events such as restaurant confirmation, food ready, driver assigned, pickup, delivery confirmation, driver cancellation, and customer cancellation.

3. Transition tests

Test every valid transition and reject invalid transitions—for example, Delivered must not return to Preparing. Test driver cancellation before pickup, during delivery, repeated cancellation events, network delay, duplicate event delivery, and cancellation arriving after delivery confirmation. Verify the user interface, backend order status, notifications, and refund/reassignment behavior stay consistent.

4. Coverage criteria

Track state coverage, transition coverage, and important transition-pair coverage. Include timeout and recovery paths and test simultaneous or out-of-order events.

ECP and BVA focus on input classes and numeric limits; they may not reveal sequence-dependent workflow defects. State transition testing directly validates legal and illegal sequences, making it appropriate for dynamic order tracking.

Q11. Cybersecurity Authentication: Hybrid Testing

1.Brute-force vulnerabilities suggest that authentication testing did not adequately verify attempt limits, throttling, lockout, OTP handling, or interactions between password, location, and timing logic.

2. Black-box and error-guessing tests

Test repeated incorrect passwords, repeated and expired OTPs, OTP replay, missing fields, account enumeration, concurrent attempts, password reset abuse, and requests from unusual IP locations. Verify consistent error messages and that rate limits apply across sessions and relevant endpoints. Conduct testing only in an authorized environment.

3. White-box path analysis

Construct a CFG for credential validation, OTP generation/verification, time-window checks, IP/risk evaluation, attempt counting, lockout, and success/failure logging. Test paths for correct and incorrect passwords, valid/expired/replayed OTP, allowed/high-risk IP, lockout threshold, and recovery. Inspect whether any path bypasses rate limiting or resets counters incorrectly.

4. Security controls

Use per-account and per-IP throttling, progressive delays or temporary lockout with safeguards against denial-of-service abuse, secure OTP expiry and single-use validation,

strong password storage, MFA enforcement, monitoring, and alerting. Avoid revealing whether a username exists.

Error guessing targets common attack patterns; path analysis checks internal enforcement paths and bypasses. Combine these with automated security tests, code review, and authorized penetration testing. No single coverage metric guarantees security.

Q12. E-Learning Subscription Expiration and Upgrades

1.Errors on the exact expiration date suggest ambiguous date semantics, time-zone handling, or incorrect inclusive/exclusive comparisons. The product rules must state whether access ends at the start or end of the expiration date and how upgrades take effect.

2. Equivalence classes

Partition subscriptions into active, expiring today, expired, not-yet-started, and invalid date ranges. Partition plan types into Free, Basic, and Premium, and course types into those allowed and not allowed for each plan. Include valid and invalid upgrade/downgrade combinations.

3. Boundary tests

Test immediately before expiration, exactly at the defined expiration instant, and immediately after it. Test upgrades submitted on the expiration date, across midnight, and across relevant time zones. Use the system's specified time zone and timestamp precision.

4. Expected behavior

Verify access rights, billing, renewal dates, course availability, and confirmation messages. Ensure a failed upgrade does not leave the account in a partially updated state. Add tests for delayed payment confirmation and repeated upgrade requests if supported.

ECP reduces the date and plan space to meaningful classes; BVA targets the exact transition where defects occur. A decision table can map plan, date status, course type, and payment status to the expected access result. This gives systematic, auditable coverage of subscription transitions.

Q13. Smart-Home Lighting: Branch and Condition Coverage

1.Statement coverage may execute each line without evaluating every switch-case outcome or proving that each Boolean condition affects the final decision. This can leave lighting and dimming errors undetected.

2. White-box model

Inspect switch statements for every supported sensor mode, dimming level, and error/default case. For each compound Boolean expression, identify atomic conditions and their true/false outcomes. Build a CFG including sensor reading, validation, mode selection, dimming calculation, actuator command, and error handling.

3. Test design

Use branch coverage to exercise every decision outcome and switch case, including default/unsupported values. Use condition coverage to make each atomic condition true and false. Where decisions are safety- or reliability-critical, add MC/DC pairs so each condition independently affects the decision. Test sensor boundaries, missing/stuck readings, rapid ambient-light changes, and actuator communication failures.

4. Hardware-software integration

Check that calculated dimming levels map correctly to PWM or actuator commands, outputs remain within hardware limits, and invalid sensor data triggers the approved fallback. Include hardware-in-the-loop tests and verify timing and response to changing inputs.

Branch and condition coverage expose untested logical outcomes that statement coverage can miss. Integration tests then validate that correct software decisions produce correct physical behavior. The combination reduces, but does not eliminate, the risk of hardware-software failures.

Q14. Logistics Shipping: Zero and Negative Weights

1.Failure to test zero and negative weights allowed invalid inputs to reach shipping-cost calculations. This can cause free or negative shipping charges, incorrect invoices, and downstream accounting errors.

2. Robust Boundary Value Analysis

Confirm the valid weight range is Wmin through Wmax. Test $W_{min}-\varepsilon$, Wmin, $W_{min}+\varepsilon$, a typical valid weight, $W_{max}-\varepsilon$, Wmax, and $W_{max}+\varepsilon$. If zero and negative weights can be entered, explicitly test negative values, zero, and the smallest valid positive value. Choose ε according to the supported unit and precision.

3. Other input dimensions

Repeat relevant tests across destination zones and delivery speeds, including unknown zones and unsupported service levels. Check empty input, nonnumeric text, excessive decimal places, very large values, and unit-conversion errors. Verify that invalid weight is rejected before cost calculation.

4. Expected results

For invalid values, expect a clear validation error and no shipment or invoice to be created. For valid boundaries, compare the fee against the approved rate table and check rounding, taxes, and currency precision.

Robust BVA deliberately includes values outside the valid range, where input validation defects are common. Negative scenarios are essential because normal-value tests cannot show whether the system rejects invalid data safely.

Q15. Medical Imaging: Prime-Size Parallel Processing Defects

1.Errors occurring only for prime-sized datasets suggest a size-dependent partitioning, chunking, indexing, synchronization, or remainder-handling defect. Random testing alone may miss prime sizes or fail to reproduce the triggering execution.

2. Basis-path testing

Construct a CFG for input validation, dataset partitioning, worker creation, parallel processing, remainder handling, synchronization, aggregation, and output validation. Identify independent paths and use cyclomatic complexity $V(G) = E - N + 2P$ (for a single connected graph, $E - N + 2$) to guide basis-path selection. Derive actual paths from the implementation.

3. Structured data sizes

Test sizes of 0 or the minimum permitted size, 1, small composite numbers, several prime numbers, sizes just below and above worker/chunk multiples, and large datasets. Include sizes that leave a remainder when divided among workers, as well as sizes evenly divisible by the worker count.

4. Accuracy and stability checks

Compare parallel outputs against a trusted serial reference implementation or validated expected results. Check that every item is processed exactly once, no items are omitted or duplicated, results are deterministic where required, and synchronization avoids race conditions. Repeat tests under different worker counts and scheduling conditions.

5. Role of random testing

Retain seeded randomized and stress tests as supplementary tests, recording seeds and configurations for reproducibility. They should not replace structured path and boundary testing. Combine basis-path tests with prime/composite sizes, concurrency testing, memory checks, and regression tests for the defect.

Basis-path testing systematically exercises independent control-flow logic, while targeted dataset sizes probe the suspected mathematical and partitioning edge cases. Reference comparisons and concurrency tests provide stronger evidence of accuracy and stability than random testing alone.